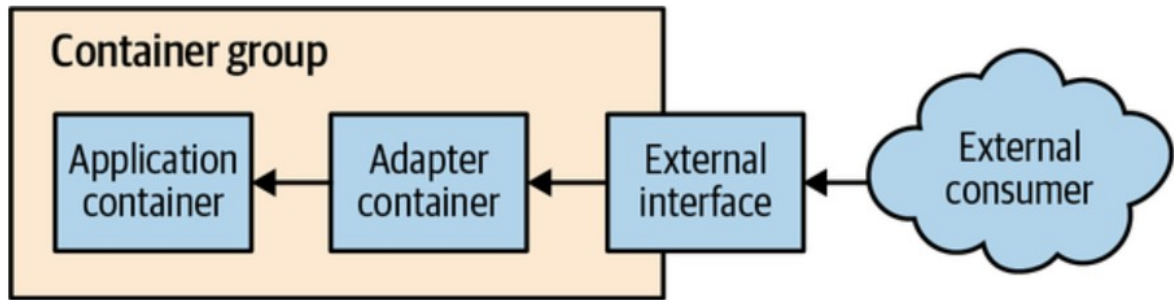


Patrón Adapter

En este ejemplo se usa el patrón Adapter para implementar un monitor de salud para una BD de MySQL.

El ejemplo es **Hands On: Adding Rich Health Monitoring for MySQL** (Libro: Designing Distributed Systems, 2nd Edition). La arquitectura del patrón se observa a continuación:



Conceptos esenciales para entender el ejercicio

Pods

Archivos necesarios

Dockerfile, main.go, pod-adapter.yaml, script.sh, forwardPort.sh

Repositorio de GitHub

<https://github.com/SalomonAvila/Introduccion-Distribuidos/tree/main/Adapter>

Notas sobre la ejecución

el ejercicio fue probado en minikube, en un ambiente con la distribución Ubuntu entregado en máquina virtual de la universidad.

Cómo se ejecuta el taller:

1. Iniciar Minikube
2. Crear la imagen de Docker.

```
docker build -t mi-adapter-local .
```

Con este comando se crea una imagen de Docker con nombre personalizado "mi-adapter-local".

3. Cargar la imagen de Docker a minikube, dejándola en local y evitando la necesidad de u

```
minikube image load mi-adapter-local
```

4. Desplegar el pod:

```
kubectl delete pod adapter-health
```

```
kubectl apply -f pod-adapter.yaml
```

Resultado

```
kubectl get pods
```

NAME	READY	STATUS	RESTARTS	AGE
adapter-health	2/2	Running	0	42s

5. Crear un túnel entre minikube y la máquina

```
kubectl port-forward adapter-health 8080:8080
```

Resultado accesible en: <http://localhost:8080/healthz>

Note que durante el tiempo que se ejecuta la redirección del puerto, la terminal queda ocupada, por lo tanto, debe correr otra terminal en caso de querer seguir trabajando en ella.

Alternamente, puede asignarle permisos de administrador a los scripts (usando `chmod +x script.sh` y `chmod +x forwardPort.sh`)

Realizado por

Salomon Alfredo Avila Larrota